



Reporte de Pruebas Unitarias

Integrantes (Programangos):

Carlos Daniel Albarracín Cruz

Pablo Andrés Pérez Beltrán

David Nickolai Parra Ariza

Jhoan Sebastián Yaya García

Docente: Oscar Eduardo Álvarez Rodríguez

Fecha: 14/06/2026

Curso:

2016701: Ingeniería de Software I

Facultad de Ingeniería, Universidad Nacional de Colombia

1. Introducción

El presente reporte describe las pruebas unitarias implementadas para el proyecto SISA (Sistema de Supervivencia Académica).

Las pruebas fueron desarrolladas utilizando **pytest** como framework principal de testing, junto con **unittest.mock** para el aislamiento de dependencias externas. Esto permite validar la lógica de negocio de cada módulo de forma completamente independiente de la base de datos y de la capa HTTP.

La arquitectura del backend sigue un patrón de capas (domain, infra, services, controllers), por lo que las pruebas se enfocan exclusivamente en la capa de servicios (**services/**), que es donde reside la lógica de negocio central del sistema. En cada prueba, el repositorio es reemplazado por un objeto **mock**, garantizando que los tests sean: rápidos, deterministas y no dependan de infraestructura externa.

Cada integrante del equipo fue responsable de un módulo distinto del sistema, **cubriendo tres requisitos funcionales por módulo**. En total se implementaron 28 pruebas unitarias distribuidas entre los cuatro módulos: Consejos, Autenticación, Trámites y Avisos.

La herramienta de cobertura utilizada fue **coverage.py**, que permite medir el porcentaje de líneas de código ejecutadas durante la ejecución de los tests. El proyecto alcanzó una cobertura global del 95 %, lo cual refleja una validación exhaustiva de los casos de uso principales y sus casos límite.

2. Módulo Consejos

Integrante: David Nickolai Parra Ariza

Requisitos cubiertos: RF_10 (publicar consejo), RF_11 (like a consejo), RF_12 (reportar consejo)

Servicio testado: AdvicesService

Test	Requisito	Descripción	Resultado esperado
test_crear_consejo_valido	RF_10	Publica un consejo con título, contenido y categoría validos	Retorna el objeto creado con id=1
test_rechazar_consejo_sin_titulo	RF_10	Intenta publicar un consejo con título vacío	ValidationError: título obligatorio
test_rechazar_consejo_sin_contenido	RF_10	Intenta publicar un consejo con contenido vacío	ValidationError: contenido obligatorio
test_rechazar_consejo_sin_categoria	RF_10	Intenta publicar un consejo con categoría vacía	ValidationError: categoría obligatoria
test_rechazar_like_duplicado	RF_11	Usuario intenta dar like a un consejo que ya voto	ValidationError: usuario ya votó
test_crear_like_valido	RF_11	Usuario da like a un consejo que no ha votado	Retorna el like creado con advice_id=1

3. Modulo Autenticación

Integrante: Carlos Daniel Albarracín Cruz

Requisitos cubiertos: RF_01 (registro), RF_03 (login), RF_05 (editar perfil)

Servicio testado: AuthService

Test	Requisito	Descripción	Resultado esperado
test_registro_email_valido	RF_01	Registra un usuario con correo institucional @unal.edu.co válido	Retorna usuario creado con el email correcto
test_registro_email_no_institucional	RF_01	Intenta registrar con correo @gmail.com	ValidationError: correo institucional
test_registro_email_duplicado	RF_01	Intenta registrar un correo ya existente en el sistema	ValidationError: correo ya registrado
test_login_contrasena_incorrecta	RF_03	Usuario existe, pero la contraseña enviada no coincide con el hash	ValidationError: credenciales inválidas
test_login_usuario_no_existe	RF_03	Intenta login con un correo que no existe en el sistema	ValidationError: credenciales inválidas
test_actualizar_perfil_campos_validos	RF_05	Actualiza major y current_semester de un usuario existente	Retorna usuario con major='Sistemas'

4. Modulo Trámites

Integrante: Pablo Andrés Pérez Beltrán

Requisitos cubiertos: RF_14 (registrar experiencia), RF_16 (like a experiencia)

Servicio testado: ProceduresService

Test	Requisito	Descripción	Resultado esperado
test_crear_experiencia_valida	RF_14	Registra experiencia con comentario y tiempo en días mayor a cero	Retorna experiencia creada con id=1
test_rechazar_tiempo_negativo	RF_14	Intenta registrar experiencia con tiempo de duración negativo	ValidationError: tiempo mayor a cero
test_rechazar_tiempo_cero	RF_14	Intenta registrar experiencia con tiempo igual a cero	ValidationError: tiempo mayor a cero
test_rechazar_comentario_vacio	RF_14	Intenta registrar experiencia con comentario vacío	ValidationError: comentario obligatorio
test_rechazar_voto_duplicado	RF_16	Usuario intenta votar una experiencia que ya votó	ValidationError: usuario ya votó
test_crear_voto_valido	RF_16	Usuario vota una experiencia que no ha votado previamente	Retorna voto con experience_id=1

5. Modulo Avisos

Integrante: Jhoan Sebastián Yaya García

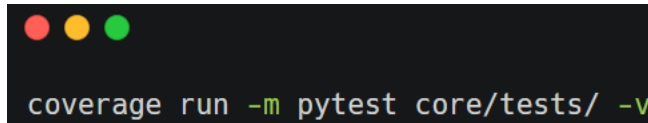
Requisitos cubiertos: RF_21 (publicar aviso), RF_22 (like a aviso), RF_23 (reportar aviso)

Servicio testado: NoticesService

Test	Requisito	Descripción	Resultado esperado
test_crear_aviso_valido	RF_21	Publica aviso con título válido y fecha de expiración futura	Retorna aviso creado con id=1
test_rechazar_aviso_sin_titulo	RF_21	Intenta publicar aviso con título vacío	ValidationError: título obligatorio
test_rechazar_aviso_titulo_espacios	RF_21	Intenta publicar aviso con título de solo espacios en blanco	ValidationError: título obligatorio
test_rechazar_fecha_pasada	RF_21	Intenta publicar aviso con fecha de expiración en el pasado	ValidationError: fecha en el pasado
test_crear_aviso_sin_fecha_expiracion	RF_21	Publica aviso sin especificar fecha de expiración (campo opcional)	Retorna aviso creado con id=2
test_rechazar_like_duplicado	RF_22	Usuario intenta dar like a un aviso que ya votó	ValidationError: usuario ya votó
test_crear_like_valido	RF_22	Usuario da like a un aviso que no ha votado previamente	Retorna like con notice_id=1
test_reportar_aviso_valido	RF_23	Reporta un aviso con una razón de reporte válida	Retorna reporte con status='pending'
test_rechazar_reporte_sin_razon	RF_23	Intenta reportar aviso con razón vacía	ValidationError: razón obligatoria
test_rechazar_reporte_razon_espacios	RF_23	Intenta reportar aviso con razón de solo espacios en blanco	ValidationError: razón obligatoria

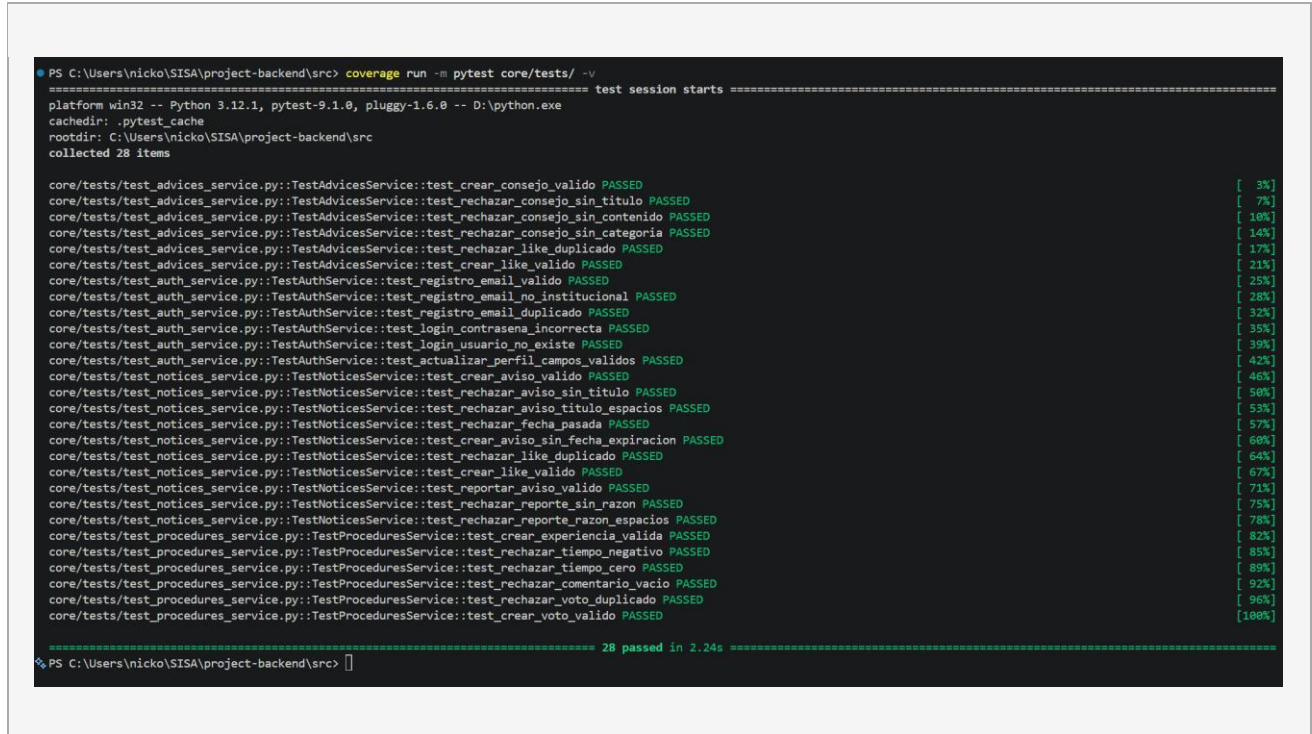
6. Resultados de Ejecución

Los 28 tests fueron ejecutados con el siguiente comando desde la carpeta **src/**:



```
coverage run -m pytest core/tests/ -v
```

Figura 1. Resultado de ejecución de los 28 tests (*pytest -v*)



```
PS C:\Users\nicko\SISA\project-backend\src> coverage run -m pytest core/tests/ -v
===== test session starts =====
platform win32 -- Python 3.12.1, pytest-9.1.0, pluggy-1.6.0 -- D:\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\nicko\SISA\project-backend\src
collected 28 items

core/tests/test_advice_service.py::TestAdviceService::test_crear_consejo_valido PASSED [ 3%]
core/tests/test_advice_service.py::TestAdviceService::test_rechazar_consejo_sin_titulo PASSED [ 7%]
core/tests/test_advice_service.py::TestAdviceService::test_rechazar_consejo_sin_contenido PASSED [ 10%]
core/tests/test_advice_service.py::TestAdviceService::test_rechazar_consejo_sin_categoria PASSED [ 14%]
core/tests/test_advice_service.py::TestAdviceService::test_rechazar_like_duplicado PASSED [ 17%]
core/tests/test_advice_service.py::TestAdviceService::test_crear_like_valido PASSED [ 21%]
core/tests/test_auth_service.py::TestAuthService::test_registro_email_valido PASSED [ 25%]
core/tests/test_auth_service.py::TestAuthService::test_registro_email_no_institucional PASSED [ 28%]
core/tests/test_auth_service.py::TestAuthService::test_registro_email_duplicado PASSED [ 32%]
core/tests/test_auth_service.py::TestAuthService::test_login_contrasena_incorrecta PASSED [ 35%]
core/tests/test_auth_service.py::TestAuthService::test_login_usuario_no_existe PASSED [ 39%]
core/tests/test_auth_service.py::TestAuthService::test_actualizar_perfil_campos_validos PASSED [ 42%]
core/tests/test_notices_service.py::TestNoticesService::test_crear_aviso_valido PASSED [ 46%]
core/tests/test_notices_service.py::TestNoticesService::test_rechazar_aviso_sin_titulo PASSED [ 50%]
core/tests/test_notices_service.py::TestNoticesService::test_rechazar_aviso_titulo_espacios PASSED [ 53%]
core/tests/test_notices_service.py::TestNoticesService::test_rechazar_fecha_pasada PASSED [ 57%]
core/tests/test_notices_service.py::TestNoticesService::test_crear_aviso_sin_fecha_expiration PASSED [ 60%]
core/tests/test_notices_service.py::TestNoticesService::test_rechazar_like_duplicado PASSED [ 64%]
core/tests/test_notices_service.py::TestNoticesService::test_crear_like_valido PASSED [ 67%]
core/tests/test_notices_service.py::TestNoticesService::test_reportar_aviso_valido PASSED [ 71%]
core/tests/test_notices_service.py::TestNoticesService::test_rechazar_reporte_sin_razon PASSED [ 75%]
core/tests/test_notices_service.py::TestNoticesService::test_rechazar_reporte_razon_espacios PASSED [ 78%]
core/tests/test_procedures_service.py::TestProceduresService::test_crear_experiencia_valida PASSED [ 82%]
core/tests/test_procedures_service.py::TestProceduresService::test_rechazar_tiempo_negativo PASSED [ 85%]
core/tests/test_procedures_service.py::TestProceduresService::test_rechazar_tiempo_cero PASSED [ 89%]
core/tests/test_procedures_service.py::TestProceduresService::test_rechazar_comentario_vacio PASSED [ 92%]
core/tests/test_procedures_service.py::TestProceduresService::test_rechazar_voto_duplicado PASSED [ 96%]
core/tests/test_procedures_service.py::TestProceduresService::test_crear_voto_valido PASSED [100%]

===== 28 passed in 2.24s =====
PS C:\Users\nicko\SISA\project-backend\src>
```

Figura 2. Reporte de cobertura por módulo (*coverage report -m*)

```
PS C:\Users\nicko\SISA\project-backend\src> coverage report -m
Name                               Stmts  Miss  Cover   Missing
-----
core\__init__.py                   0      0  100%
core\services\__init__.py          0      0  100%
core\services\advices_service.py   19      2   89%   7-8
core\services\auth_service.py      28      4   86%   8-9, 28, 34
core\services\notices_service.py   24      2   92%   8-9
core\services\procedures_service.py 21      5   76%   7-8, 19-21
core\tests\__init__.py             0      0  100%
core\tests\test_advices_service.py 36      0  100%
core\tests\test_auth_service.py    37      0  100%
core\tests\test_notices_service.py 52      0  100%
core\tests\test_procedures_service.py 34      0  100%
-----
TOTAL                             251     13   95%
PS C:\Users\nicko\SISA\project-backend\src>
```

Figura 3. Reporte HTML de cobertura (*htmlcov/index.html*) - cobertura global 95%

Coverage report: 95%

Files Functions Classes

coverage.py v7.14.1, created at 2026-06-13 22:55 -0500

File ▲	statements	missing	excluded	coverage
core__init__.py	0	0	0	100%
core\services__init__.py	0	0	0	100%
core\services\advices_service.py	19	2	0	89%
core\services\auth_service.py	28	4	0	86%
core\services\notices_service.py	24	2	0	92%
core\services\procedures_service.py	21	5	0	76%
core\tests__init__.py	0	0	0	100%
core\tests\test_advices_service.py	36	0	0	100%
core\tests\test_auth_service.py	37	0	0	100%
core\tests\test_notices_service.py	52	0	0	100%
core\tests\test_procedures_service.py	34	0	0	100%
Total	251	13	0	95%

coverage.py v7.14.1, created at 2026-06-13 22:55 -0500

El proyecto alcanzó una cobertura global del 95 % sobre 251 sentencias analizadas, con únicamente 13 líneas no cubiertas correspondientes a la inicialización lazy del repositorio real en cada servicio (código que no se ejecuta cuando se inyecta el mock en los tests). Los 28 tests pasaron exitosamente en 2.24 segundos.